Question 1 Max. score: 30.00

Maximum work

Bob has N workers working under him. The i^{th} worker has some strength W_i. Currently, he has M tasks pending with him. Each task can be assigned to only 1 worker. To do particular task i, Strength S_i is required i.e. a task i can be done by a worker only if his strength is greater or equal to S_i. Bob has K magical pills, taking a magical pill will increase the strength of a worker by B units. Bob gives pills to workers optimally such that the maximum number of tasks can be done.

Task

Determine the maximum number of tasks that can be completed.

Note: 1-based indexing is followed.

Example

Assumptions

- $K = 10$
- $B = 1$
- $N = 3$
- $W = [10, 5, 4]$
- $M = 3$
- $S = [15, 4, 20]$

New Submission | All Submissions

Auto-complete connection failed. Reconnect

```cpp
#include<bits/stdc++.h>
using namespace std;

int getAns(int i, vector<int> &W,
vector<int> &S, int k, int B, int n,
vector<vector<int>> &dp)
{
    if(i==n) return 0;
    if(dp[i][k]!=-1) return dp[i][k];
    if(S[i]>=W[i]) return dp[i][k]=1+getAns(i
+1, W, S, k, B, n, dp);

    int ans= getAns(i+1, W, S, k, B, n, dp);
    int temp=S[i];
    int x=k;
    while(x && temp<W[i]){
        x--;
        temp+=B;
    }
    if(temp>=W[i])  ans=max(ans, 1+getAns(i
+1, W, S, x, B, n, dp));
    return dp[i][k]=ans;
}
int solve (int K, int B,int N, vector<int> W,
int M, vector<int> S) {
   // Write your code here
```

Provide custom input

- *M = 3*
- *S = [15, 4, 20]*

Approach

There are many ways to assign workers to different tasks, some of the ways are:

- Bob can assign task *2* to worker *2* and he can give *5* pills to worker *1* which will increase his strength to *15* and then assign task *1* to him. So at max *2* tasks can be completed. There is no way in which he can get more than *2* tasks done.
- Alternatively, he can give *10* pills to worker *3* and assign him task *3*, and assign worker *1* to task *2*.
- Therefore, the maximum number of tasks that can be assigned is 2.

Function description

Complete the *solve* function provided in the editor. This function takes the following *6* parameters and returns an integer:

- *K*: Represents the number of magical pills
- *B*: Represents an integer denoting the amount of strength that will be increased by taking a pill
- *N*: Represents the number of workers
- *W*: Represents an array of integers denoting the strength of the workers
- *M*: Represents the number of tasks
- *S*: Represents an array of integers denoting strength required for the task

Input format

Note: This is the input format that you must use to provide custom input (available above the

New Submission | All Submissions

Auto-complete connection failed. Reconnect

```
#include<bits/stdc++.h>
using namespace std;

int getAns(int i, vector<int> &W,
vector<int> &S, int k, int B, int n,
vector<vector<int>> &dp)
{
    if(i==n) return 0;
    if(dp[i][k]!=-1) return dp[i][k];
    if(S[i]>=W[i]) return dp[i][k]=1+getAns(i
+1, W, S, k, B, n, dp);

    int ans= getAns(i+1, W, S, k, B, n, dp);
    int temp=S[i];
    int x=k;
    while(x && temp<W[i]){
        x--;
        temp+=B;
    }
    if(temp>=W[i])  ans=max(ans, 1+getAns(i
+1, W, S, x, B, n, dp));
    return dp[i][k]=ans;
}
int solve (int K, int B,int N, vector<int> W,
int M, vector<int> S) {
   // Write your code here
```

Provide custom input

- *M*: Represents the number of tasks
- *S*: Represents an array of integers denoting strength required for the task

Input format

Note: This is the input format that you must use to provide custom input (available above the Compile and Test button).

- The first line contains an integer *T* denoting the number of test cases. *T* also denotes the number of times you have to run the *solve* function on a different set of inputs.
- For each test case:
  - The first line contains two space-separated integers *K* and *B*.
  - The second line contains an integer *N* denoting the number of workers.
  - The third line contains *N* space-separated integers denoting an array *W* where *W[i]* represents the strength of the i^{th} worker.
  - The fourth line contains an integer *M* denoting the number of tasks.
  - The fifth line contains *M* space-separated integers denoting an array *S*. *S[i]* represents the strength required to do the i^{th} task.

Output format

For each test case in a new line, print the answer representing the maximum number of tasks that can be done.

Constraints

$1 \leq T \leq 100$

$1 \leq K \leq 10^5$

New Submission | All Submissions

Auto-complete connection failed. Reconnect

```
#include<bits/stdc++.h>
using namespace std;

int getAns(int i, vector<int> &W,
vector<int> &S, int k, int B, int n,
vector<vector<int>> &dp)
{
    if(i==n) return 0;
    if(dp[i][k]!=-1) return dp[i][k];
    if(S[i]>=W[i]) return dp[i][k]=1+getAns(i
+1, W, S, k, B, n, dp);

    int ans= getAns(i+1, W, S, k, B, n, dp);
    int temp=S[i];
    int x=k;
    while(x && temp<W[i]){
        x--;
        temp+=B;
    }
    if(temp>=W[i])  ans=max(ans, 1+getAns(i
+1, W, S, x, B, n, dp));
    return dp[i][k]=ans;
}
int solve (int K, int B,int N, vector<int> W,
int M, vector<int> S) {
   // Write your code here
```

Provide custom input

Output format

For each test case in a new line, print the answer representing the maximum number of tasks that can be done.

Constraints

$1 \leq T \leq 100$

$1 \leq K \leq 10^5$

$1 \leq B \leq 10$

$1 \leq N, M \leq 10^4$

$1 \leq W[i], S[i] \leq 10^5$

Code snippets (also called starter code/boilerplate code)

This question has code snippets for C, CPP, Java, and Python.

Sample input 1 Copy

```
1
2 1
2
3 2
2
2 3
```

Sample output 1 Copy

```
2
```

New Submission | All Submissions

Auto-complete connection failed. Reconnect

```cpp
#include<bits/stdc++.h>
using namespace std;

int getAns(int i, vector<int> &W,
vector<int> &S, int k, int B, int n,
vector<vector<int>> &dp)
{
    if(i==n) return 0;
    if(dp[i][k]!=-1) return dp[i][k];
    if(S[i]>=W[i]) return dp[i][k]=1+getAns(i
+1, W, S, k, B, n, dp);

    int ans= getAns(i+1, W, S, k, B, n, dp);
    int temp=S[i];
    int x=k;
    while(x && temp<W[i]){
        x--;
        temp+=B;
    }
    if(temp>=W[i])  ans=max(ans, 1+getAns(i
+1, W, S, x, B, n, dp));
    return dp[i][k]=ans;
}
int solve (int K, int B,int N, vector<int> W,
int M, vector<int> S) {
   // Write your code here
```

Provide custom input

Question 2 Max. score: 50.00

Shopping Spree

Tushar is on a shopping spree, and wants to buy everything that is available, given the recent situation. :p His city can be described by a tree of **N** nodes, with nodes labeled from 1 to **N.** The tree is rooted at node **1**. At each node of the tree there is a shop selling **K** different kinds of items. Each shop has a given amount of price for each of the **K** kinds of items. Due to recent situations, lockdown has been imposed in his city by the government and all the shops are shut down. There are two types of events happening though, represented as follows:

- **1 V** This means the government orders to open all the shops in the subtree of **V.** A subtree means all nodes whose ancestor is node **V**. Note that **V** also lies in its subtree. As soon as this event happens, Tushar will go shopping with his friends, Aman and Akash. They want to buy each of the **K** kinds of items available, and since they are always focused on optimization, they want to **minimize** the total cost doing so. There is no restriction on the number of items bought from a single shop. i.e. they can even buy all the items from a single shop as well as buy no item from a particular shop. You have to find the minimum total cost to buy all **K** kind of items, only from the shops that are **currently open**. You also have to find the number of ways of buying all items at this minimum total cost. Two ways are considered different if there exists an item which is bought from a different shop in both of them. **After their shopping gets over, all shops are closed once again**.
- **2 V i val** This means that the new price of the **ith** kind of item at shop **V** is **val**

You have to help Tushar and his friends do the shopping optimally for each event of type 1, with all

New Submission | All Submissions

Auto-complete connection failed. Reconnect

```c
#include<stdio.h>
#include<stdbool.h>
#include<malloc.h>

long long** Solve (int N, int K, int Q,
int** price, int* edges, int** queries,
int* out_x, int* out_y) {
    // Write your code here
    static long long result[0][0];
    *out_x = 0;
    *out_y = 0;

    return result;
}

int main() {
    int out_x;
    int out_y;
    int N;
    scanf("%d", &N);
    int K;
    scanf("%d", &K);
    int Q;
    scanf("%d", &Q);
    int i_price, j_price;
    int **price = (int **)malloc((N)*sizeof
```

Provide custom input

- **2 V i val** This means that the new price of the **ith** kind of item at shop **V** is **val**

You have to help Tushar and his friends do the shopping optimally for each event of type 1, with all the events given in order.

Example

Consider *N = 3, k = 1,* Q = 1, *price = [[1], [2], [3]], edges = [[1,2], [2,3]]* and the given query is- [1, 1]

- The minimum cost to get 1 item from subtree of 1 is 1.
- Number of ways to achieve it is 2.

So, the answer would be [[1, 2]].

Function description

Complete the *solve* function provided in the editor. This function takes the following 6 parameters and returns a 2-D array representing the answer to each event of type 1.

- *N:* Represents the number of nodes.
- *K:* Represents the number of types of items.
- *Q*: Represents the number of events.
- *price*: Represents the initial price of each item in each node.
- *edges:* Represents the edges of the tree.
- *queries:* Represents each given event.

Input format

Note: This is the input format that you must use to provide custom input (available above the **Compile and Test** button).

New Submission | All Submissions

Auto-complete connection failed. Reconnect

```
18      scanf("%d", &N);
19      int K;
20      scanf("%d", &K);
21      int Q;
22      scanf("%d", &Q);
23      int i_price, j_price;
24      int **price = (int **)malloc((N)*sizeof
    (int *));
25      for(i_price = 0; i_price < N; i_price++)
26      {
27        price[i_price] = (int *)malloc((K)
    *sizeof(int));
28      }
29      for(i_price = 0; i_price < N; i_price++)
30      {
31        for(j_price = 0; j_price < K; j_price+
    +)
32        {
33            scanf("%d", &price[i_price]
    [j_price]);
34        }
35      }
36      int i_edges;
37      int *edges = (int *)malloc(sizeof(int)*
    (N-1));
38      for(i_edges = 0; i_edges < N-1; i_edges
```

Provide custom input

- *queries:* Represents each given event.

Input format

Note: This is the input format that you must use to provide custom input (available above the **Compile and Test** button).

- First-line contains 3 integers **N**, **K**, and **Q** representing the number of nodes, number of types of items, and number of events respectively.
- The next **N** lines contain **K** integers each, with the **jth** integer in the **ith** line representing the initial price of the **jth** type of item at the **ith** shop.
- The next $(N-1)$ lines contain two integers **U** and **V,** representing an edge in the tree between node **U** and node **V**.
- The next **Q** lines are in either of the two forms described above, representing the events in order.

Output format

- For each query of type 1, print 2 space-separated integers, the minimum cost to buy all the **K** types of items from the shops that are currently open, and the number of ways to buy with the cost being equal to the minimum cost. Since the number of ways can be large, print it modulo 10^9+7

Constraints

$2 \leq N, Q \leq 10^5$

$1 \leq i \leq K \leq 10$

New Submission | All Submissions

Auto-complete connection failed. Reconnect

```
17      int N;
18      scanf("%d", &N);
19      int K;
20      scanf("%d", &K);
21      int Q;
22      scanf("%d", &Q);
23      int i_price, j_price;
24      int **price = (int **)malloc((N)*sizeof
(int *));
25      for(i_price = 0; i_price < N; i_price++)
26      {
27        price[i_price] = (int *)malloc((K)
*sizeof(int));
28      }
29      for(i_price = 0; i_price < N; i_price++)
30      {
31        for(j_price = 0; j_price < K; j_price+
+)
32        {
33            scanf("%d", &price[i_price]
[j_price]);
34        }
35      }
36      int i_edges;
37      int *edges = (int *)malloc(sizeof(int)*
(N-1));
38      for(i_edges = 0; i_edges < N-1; i_edges
```

Provide custom input

Constraints

$2 \leq N, Q \leq 10^5$

$1 \leq i \leq K \leq 10$

$1 \leq U, V \leq N$

$1 \leq price[i][j], val \leq 10^9$

Code snippets (also called starter code/boilerplate code)

This question has code snippets for C, CPP, Java, and Python.

Sample input 1 Copy

```
6 3 5
3 7 1
1 2 4
2 5 9
6 1 3
3 3 7
9 7 1
1 2
1 3
2 4
```

Sample output 1 Copy

```
3 2
3 3
5 1
17 1
```

View more

Explanation

In the first event, all shops are open. The optimal way to buy all 3 items is buy item 1 from shop 2, buy item 2 from shop 4, and buy item 3 from shops 1 or 6. The minimum cost achieved is 1 + 1 + 1 = 3. And there are two ways of

New Submission | All Submissions

Auto-complete connection failed. Reconnect

```
17      int N;
18      scanf("%d", &N);
19      int K;
20      scanf("%d", &K);
21      int Q;
22      scanf("%d", &Q);
23      int i_price, j_price;
24      int **price = (int **)malloc((N)*sizeof
        (int *));
25      for(i_price = 0; i_price < N; i_price++)
26      {
27        price[i_price] = (int *)malloc((K)
        *sizeof(int));
28      }
29      for(i_price = 0; i_price < N; i_price++)
30      {
31        for(j_price = 0; j_price < K; j_price+
        +)
32        {
33            scanf("%d", &price[i_price]
        [j_price]);
34        }
35      }
36      int i_edges;
37      int *edges = (int *)malloc(sizeof(int)*
        (N-1));
38      for(i_edges = 0; i_edges < N-1; i_edges
```

Provide custom input

Question 3 Max. score: 20.00

Tree Magic

You are given an undirected tree containing **N** nodes. The tree is rooted at **1**.

Each node has an assigned value. Value of ith node is denoted by $val[i]$. For each node, you have to find the **Magic Number**.

Magic Number of a node **u** is the number of factors of bitwise xor of the values of **Special Nodes** which are **not** in the subtree of the **u**.

An ith node is called a **Special Node** if $val[i]$ is a prime number.

Note :-

- A subtree of a node **u** is the set of nodes which are the descendants of the node **u** including **u** itself.
- Number of factors of a number must include **1** and the number itself.
- Assume number of factors of **0** as **0**.

Input Format:

The first line contains a single integer **N**. The second line contains **N** integers denoting the value of each node separated by spaces, ith value denotes $val[i]$. The third line contains **N-1** integers separated by spaces, ith value denoting the parent of $(i+1)th$ node.

Output Format:

New Submission | All Submissions

Auto-complete connection failed. Reconnect

```c
#include<stdio.h>
#include<stdbool.h>
#include<malloc.h>

int* solve (int* parent, int* val , int* out_n) {
    // Write your code here
    static int result[1] = {0};
    *out_n = 1; // size of result array

    return result;
}

int main() {
    int out_n;
    int N;
    scanf("%d", &N);
    int i_val;
    int *val = (int *)malloc(N*sizeof(int));
    for(i_val=0; i_val<N; i_val++)
      scanf("%d", &val[i_val]);
    int i_parent;
    int *parent = (int *)malloc((N-1)*sizeof(int));
    for(i_parent=0; i_parent<N-1; i_parent++)
      scanf("%d", &parent[i_parent]);

    int* out_ = solve(parent, val, &out_n);
```

Provide custom input

Output Format:

Print a single line containing the **Magic Number** of each node starting from **1** to **n**, separated by spaces.

Constraints:

- $1 \leq N \leq 100000$
- $1 \leq val[i] \leq 1000000$

Sample input 1 Copy

```
5
7 6 4 1 3
1 1 3 3
```

Sample output 1 Copy

```
0 3 2 3 2
```

Explanation

For node 1 there are no Special Nodes which are not in the subtree of node 1 so the xor value is 0

New Submission | All Submissions

Auto-complete connection failed. Reconnect

```c
#include<stdio.h>
#include<stdbool.h>
#include<malloc.h>

int* solve (int* parent, int* val , int* out_n) {
    // Write your code here
    static int result[1] = {0};
    *out_n = 1; // size of result array

    return result;
}

int main() {
    int out_n;
    int N;
    scanf("%d", &N);
    int i_val;
    int *val = (int *)malloc(N*sizeof(int));
    for(i_val=0; i_val<N; i_val++)
      scanf("%d", &val[i_val]);
    int i_parent;
    int *parent = (int *)malloc((N-1)*sizeof(int));
    for(i_parent=0; i_parent<N-1; i_parent++)
      scanf("%d", &parent[i_parent]);

    int* out_ = solve(parent, val, &out_n);
```